

BMI 500 Profiling Exercise Report

Scanpy PBMC3K / PBMC6K / PBMC10K

Heng Sun

September 2026

Scope. This report follows the BMI 500 profiling exercise requirements: coarse-grain profiling with `/usr/bin/time -v`, instrumented timing of each `###` code section with `time.time()`, fine-grain cProfile analysis of `sc.tl.rank_genes_groups`, Slurm batch execution, runtime-growth plots, bottleneck analysis, and a 20K runtime estimate.

1. Experimental setup and profiling methods

The Scanpy PBMC workflow was executed on the BMI computational cluster using Slurm. Each of the three batch jobs requested 4 GB of memory and one CPU (`--cpus-per-task=1`), and the command used `--num-threads 1`. The Python environment reported Scanpy 1.10.2, AnnData 0.12.19, NumPy 1.26.4, SciPy 1.17.1, pandas 2.3.3, scikit-learn 1.9.1, igraph 0.11.6, and louvain 0.8.2.

Profiling was performed at three levels: (1) coarse-grain whole-program measurements with `/usr/bin/time -v`; (2) section-level elapsed times using `time.time()` around every `###` section; and (3) cProfile on the final `sc.tl.rank_genes_groups(..., method="wilcoxon")` step.

2. Slurm batch scripts

The course requires a batch script for each dataset. The following are the exact Slurm batch scripts used for the pbmc3k, pbmc6k, and pbmc10k profiling runs.

pbmc3k.sh

```
#!/bin/bash -l
#SBATCH -J pbmc3k_profile
#SBATCH -p overflow
#SBATCH --mem=4G
#SBATCH --cpus-per-task=1
#SBATCH --time=1:00:00
#SBATCH --output=./slurm_outputs/pbmc3k_%j.out

source ~/miniconda3/etc/profile.d/conda.sh
conda activate bmi500

cd ~/bmi500

/usr/bin/time -v python scanpy_pbmc.py \
  --data-dir data \
  --data-set pbmc3k \
  --out-dir data \
  --num-threads 1
```

pbmc6k.sh

```
#!/bin/bash -l
#SBATCH -J pbmc6k_profile
#SBATCH -p overflow
#SBATCH --mem=4G
```

```
#SBATCH --cpus-per-task=1
#SBATCH --time=1:00:00
#SBATCH --output=./slurm_outputs/pbmc6k_%j.out

source ~/miniconda3/etc/profile.d/conda.sh
conda activate bmi500

cd ~/bmi500

/usr/bin/time -v python scanpy_pbmc.py \
  --data-dir data \
  --data-set pbmc6k \
  --out-dir data \
  --num-threads 1
```

pbmc10k.sh

```
#!/bin/bash -l
#SBATCH -J pbmc10k_profile
#SBATCH -p overflow
#SBATCH --mem=4G
#SBATCH --cpus-per-task=1
#SBATCH --time=1:00:00
#SBATCH --output=./slurm_outputs/pbmc10k_%j.out

source ~/miniconda3/etc/profile.d/conda.sh
conda activate bmi500

cd ~/bmi500

/usr/bin/time -v python scanpy_pbmc.py \
  --data-dir data \
  --data-set pbmc10k \
  --out-dir data \
  --num-threads 1
```

3. Coarse-grain profiling results

Dataset	Wall time (s)	CPU	Max RSS (KB)	Max RSS (GiB)
PBMC3K	22.64	96%	935,276	0.89
PBMC6K	25.07	101%	953,356	0.91
PBMC10K	95.53	98%	1,886,536	1.80

The PBMC3K and PBMC6K whole-program runtimes were similar (22.64 s and 25.07 s), while PBMC10K increased to 95.53 s. Maximum resident memory approximately doubled at 10K, from about 0.9 GiB to 1.8 GiB. CPU utilization remained near one fully utilized core in all runs.

4. Instrumented section-level profiling

Code section	3K (s)	6K (s)	10K (s)
Imports	2.381	1.792	3.355
Settings	8.507	4.683	6.630
Argument parsing	0.001	0.001	0.001
Data loading	0.107	0.663	9.973
Basic filtering	0.041	0.046	2.374
Normalization	0.009	0.013	0.606
Highly variable genes	0.825	0.023	1.026
Scaling	0.030	0.058	0.744
Report adata section	0.000	0.000	0.000
PCA	0.179	0.347	1.743
Neighbors	3.106	3.287	30.527
PAGA/commented section	0.000	0.000	0.000
Louvain clustering	0.082	0.242	0.924
UMAP	5.296	10.233	14.139
Write output	0.105	0.164	0.948
Rank genes	1.332	2.772	21.464

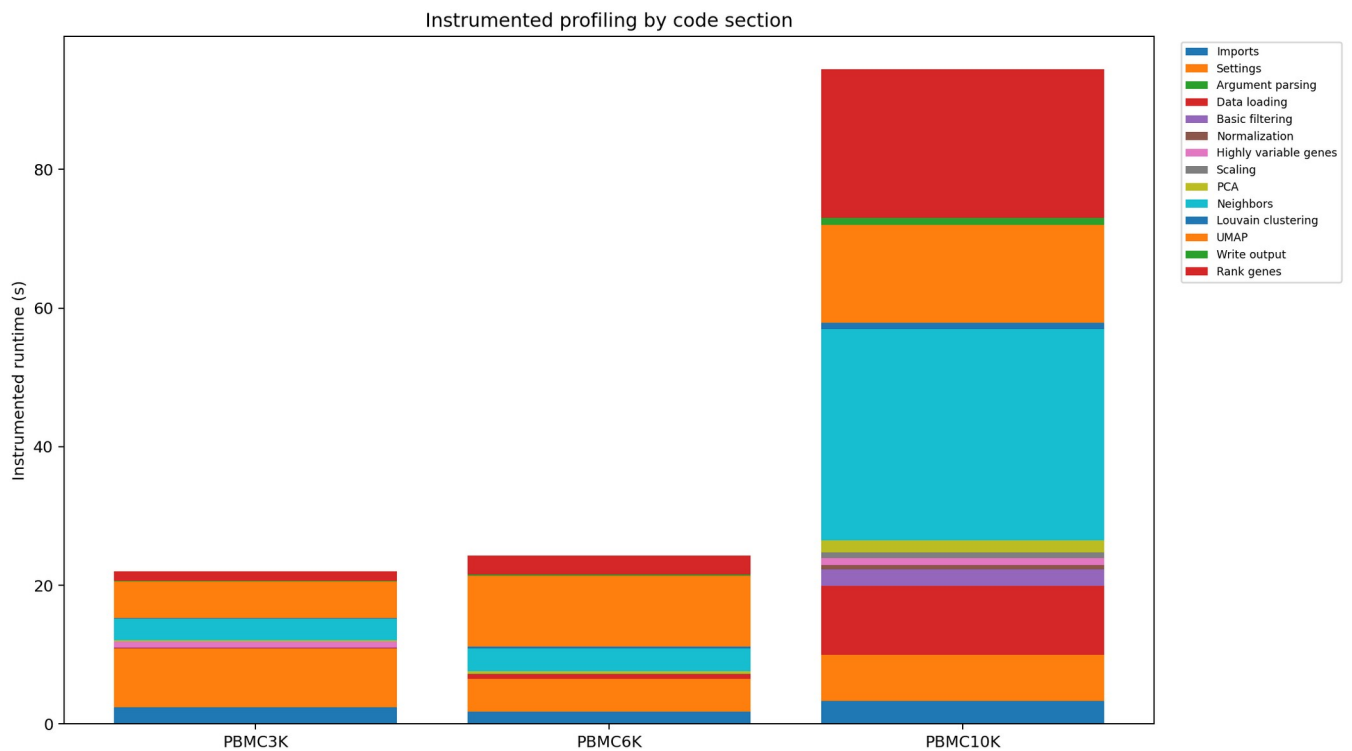


Figure 1. Instrumented runtime by code section for the three datasets.

The major bottleneck changes with dataset size. For PBMC3K, UMAP is largest (5.296 s), followed by Neighbors (3.106 s) and Rank genes (1.332 s). For PBMC6K, UMAP remains largest (10.233 s). For PBMC10K, Neighbors becomes the largest section (30.527 s), followed by Rank genes (21.464 s) and UMAP (14.139 s).

A cache-related caveat affects data-loading comparisons: PBMC3K reported reading from an existing cache file, whereas PBMC6K and PBMC10K reported writing an h5ad cache file. Therefore, the observed loading times are not a controlled scaling measurement of raw input alone.

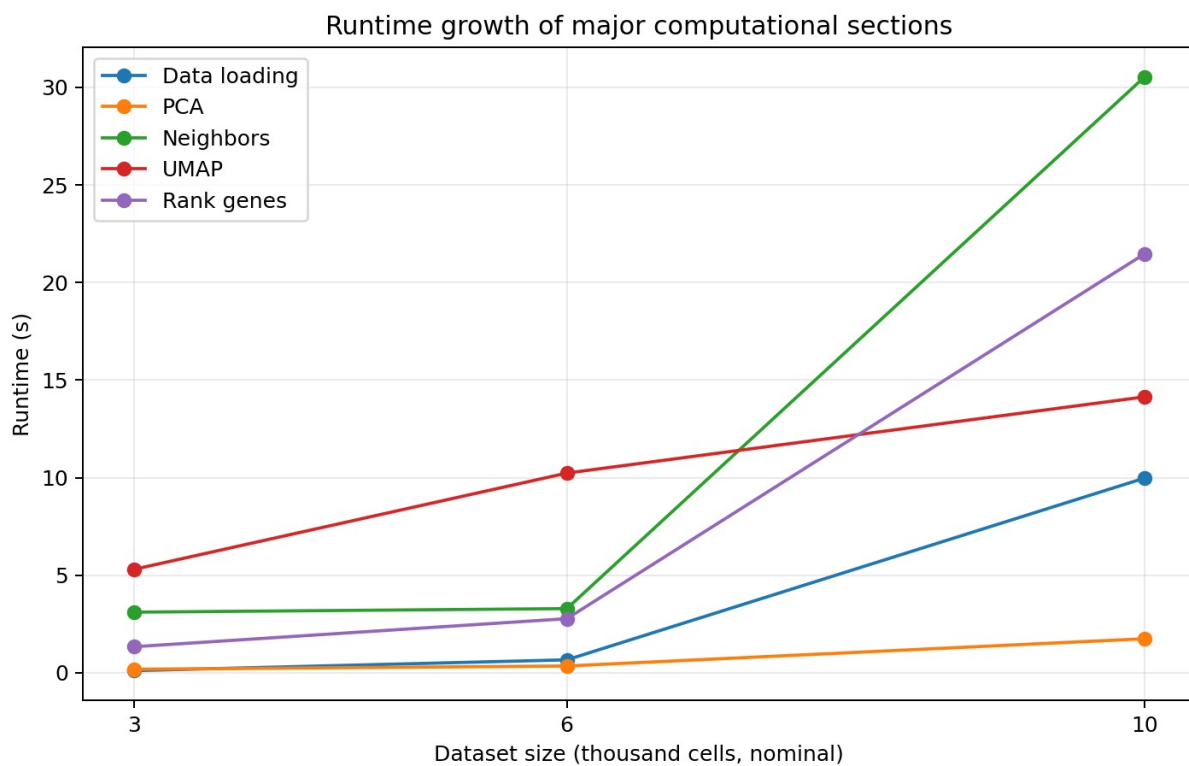


Figure 2. Runtime growth for selected computationally important sections.

5. Fine-grain cProfile analysis of rank_genes_groups

Dataset	Profile total	rank_genes_groups	pandas algorithms.rank	Sparse indexing
PBMC3K	1.327	1.326	1.000	0.065
PBMC6K	2.767	2.767	2.074	0.161
PBMC10K	21.470	20.795	11.862	5.612

Across all datasets, the Wilcoxon differential-expression path spends substantial time in ranking. In PBMC3K and PBMC6K, `pandas.core.algorithms.rank` accounts for about three quarters of the cProfile total. In PBMC10K, ranking remains the largest single cost, but sparse matrix indexing also becomes important, with `scipy.sparse._get_arrayXslice` contributing about 5.61 s cumulative time.

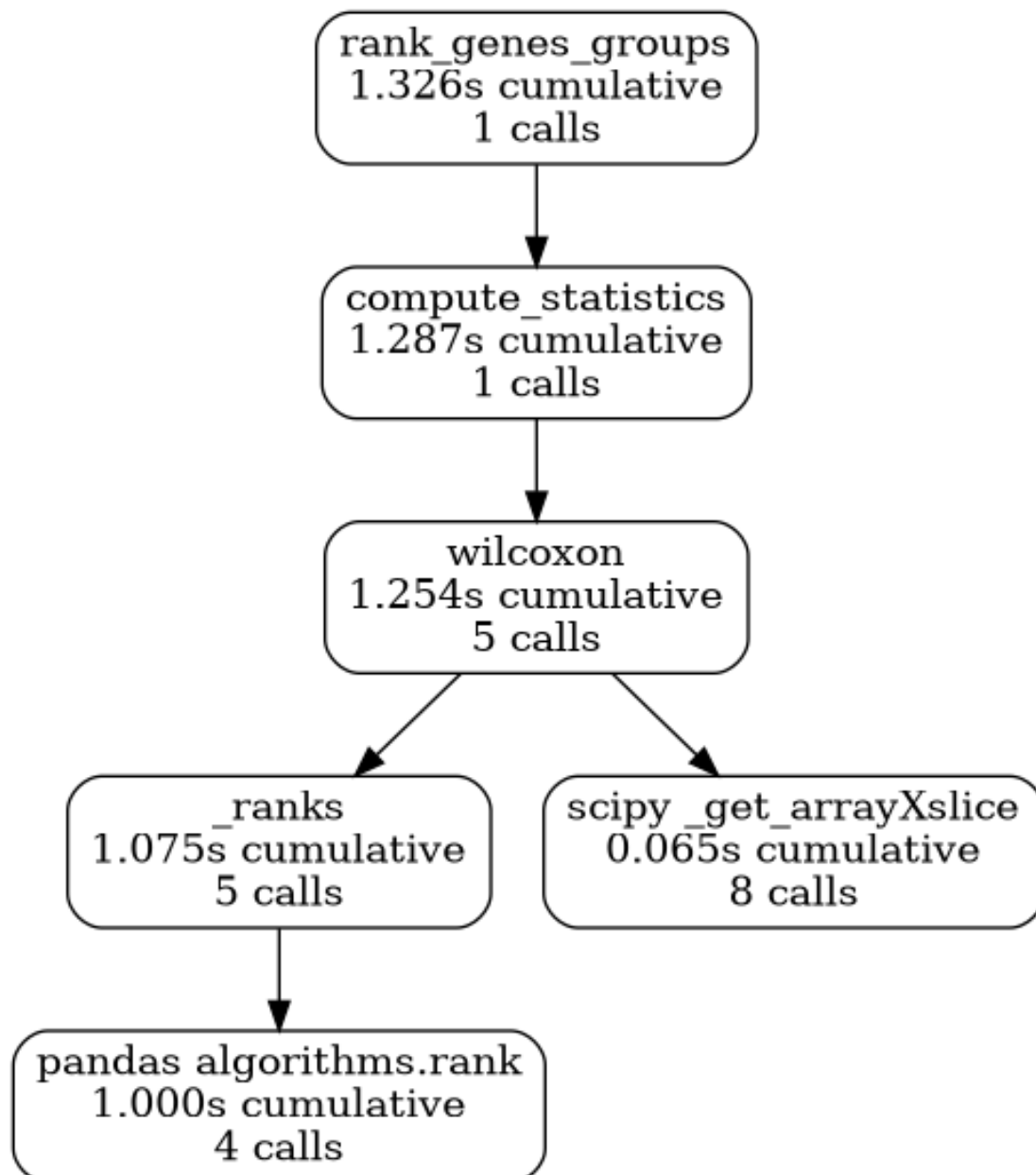


Figure 3. cProfile call-path visualization for PBMC3K rank_genes_groups.

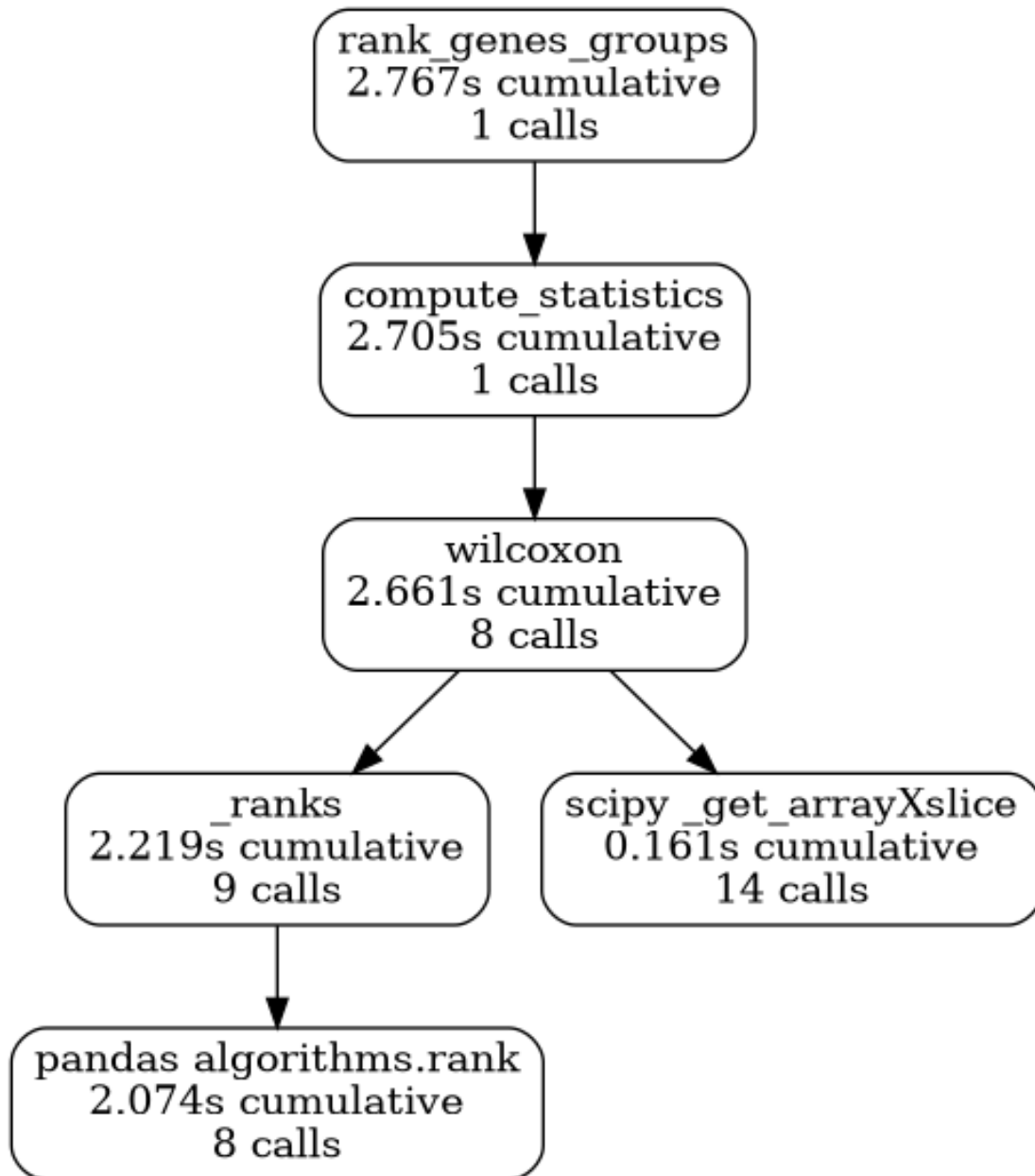


Figure 4. cProfile call-path visualization for PBMC6K rank_genes_groups.

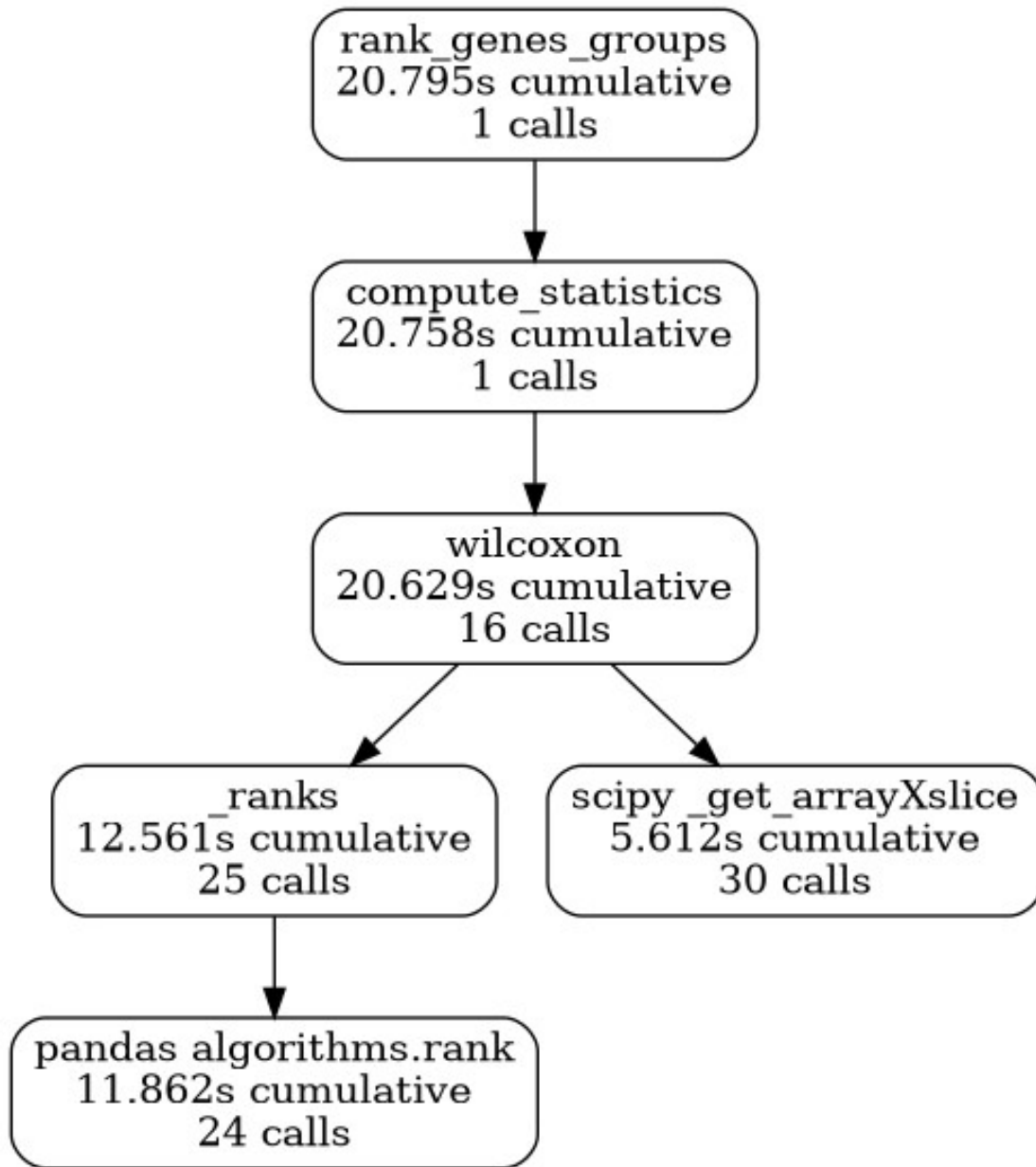


Figure 5. cProfile call-path visualization for PBMC10K rank_genes_groups.

6. Expected runtimes at 20K cells

To provide a simple, transparent extrapolation, each data-dependent section was fit with an ordinary least-squares line using the three nominal dataset sizes (3K, 6K, 10K). Imports, settings, and argument parsing were treated as startup overhead and estimated by their observed mean; zero/commented sections remain zero. These values are approximate because only three sizes were measured, the 10K run shows non-linear growth in some steps, and data loading was affected by different cache states.

Code section	Estimated 20K runtime (s)
Imports	2.509
Settings	6.607
Argument parsing	0.001
Data loading	23.522
Basic filtering	5.559
Normalization	1.423
Highly variable genes	1.182

Scaling	1.721
Report adata section	0.000
PCA	3.903
Neighbors	67.982
PAGA/commented section	0.000
Louvain clustering	2.097
UMAP	26.941
Write output	2.107
Rank genes	49.156

The summed section-level estimate is approximately 194.7 s (~3.25 min), but this should not be interpreted as a precise whole-program prediction. Neighbors (~68.0 s), Rank genes (~49.2 s), UMAP (~26.9 s), and data loading (~23.5 s, cache-sensitive) dominate the extrapolated total.

7. Performance bottlenecks and interpretation

UMAP dominates small-to-medium datasets: UMAP is the longest section at 3K and 6K. Its runtime grows from 5.30 s to 10.23 s to 14.14 s.

Neighbor graph construction becomes the largest 10K bottleneck: Neighbors is only ~3.1-3.3 s at 3K/6K but rises to 30.53 s at 10K, indicating much steeper growth in this measured range.

Rank-gene testing also scales sharply: Rank genes grows from 1.33 s to 2.77 s and then 21.46 s. cProfile attributes the largest internal cost to rank computation, with sparse indexing becoming substantial at 10K.

Memory pressure increases at 10K: Maximum RSS increases from ~0.9 GiB for 3K/6K to ~1.8 GiB for 10K. The Scanpy warning that zero-centered scaling densifies sparse input is consistent with a step that can increase memory demand.

Loading results are confounded by caching: 3K used an existing cache while 6K/10K created one, so loading-time growth should not be interpreted as a controlled algorithmic trend.

8. Console output captures (key profiling lines)

PBMC3K

```
[PROFILE] Data loading: 0.107372 seconds
[PROFILE] PCA: 0.178536 seconds
[PROFILE] Neighbors: 3.105587 seconds
[PROFILE] UMAP: 5.296170 seconds
[PROFILE] Rank genes: 1.331902 seconds
Percent of CPU this job got: 96%
Elapsed (wall clock) time: 0:22.64
Maximum resident set size (kbytes): 935276
Exit status: 0
```

PBMC6K

```
[PROFILE] Data loading: 0.662625 seconds
[PROFILE] PCA: 0.346618 seconds
[PROFILE] Neighbors: 3.286643 seconds
[PROFILE] UMAP: 10.233209 seconds
[PROFILE] Rank genes: 2.771911 seconds
Percent of CPU this job got: 101%
Elapsed (wall clock) time: 0:25.07
Maximum resident set size (kbytes): 953356
Exit status: 0
```

PBMC10K

[PROFILE] Data loading: 9.973366 seconds
[PROFILE] PCA: 1.743056 seconds
[PROFILE] Neighbors: 30.527452 seconds
[PROFILE] UMAP: 14.138627 seconds
[PROFILE] Rank genes: 21.464149 seconds
Percent of CPU this job got: 98%
Elapsed (wall clock) time: 1:35.53
Maximum resident set size (kbytes): 1886536
Exit status: 0

The full Slurm .out files also contained Scanpy progress messages and deprecation/future warnings. The profiling lines above are the console measurements required for the report and are the values analyzed in the preceding sections.

9. Conclusion

The three profiling methods provide complementary views of performance. Whole-program timing shows a large jump in runtime and memory at 10K cells. Instrumented profiling localizes the growth primarily to Neighbors, Rank genes, and UMAP. Fine-grain cProfile further shows that the Wilcoxon rank-gene step is dominated by rank computation, with sparse matrix indexing becoming a second major contributor at 10K. These measurements suggest that performance optimization for larger PBMC datasets should focus first on neighbor construction and differential-expression ranking, while controlling cache state when benchmarking I/O.

Reference to course instructions

BMI 500 Lecture 3, Profiling Exercise and Profiling Exercise Report (course slides 75-76); cProfile visualization example (course slide 64).